

MIP_PROMPT_LIBRARY_V2_PART2_PARSING.md

Mainframe Intelligence Platform

Enterprise Prompt Library V2

Part 2 - Parsing Prompts

Version: 2.0

Status: Approved

Purpose

Discovery tells us what exists.

Parsing transforms source code into structured metadata.

The parser layer is the foundation of MIP.

Without reliable parsers:

- Metadata becomes unreliable
- Knowledge graphs become inaccurate
- Impact analysis becomes dangerous
- Modernization recommendations become invalid

The goal is not code conversion.

The goal is metadata extraction.

Parser Engineering Principles

Every parser must be:

Deterministic

Same input must always produce same output.

Explainable

Every extracted fact must point to source evidence.

Extensible

Support future language variants.

Fault Tolerant

Continue processing even when source contains errors.

Observable

Provide detailed extraction metrics.

Prompt 06

COBOL Parser Architecture

File

prompts/parsing/cobol_parser.md

Purpose

Design a production-grade COBOL parser capable of extracting metadata from enterprise COBOL applications.

Context

COBOL programs contain:

- Program definitions
- Business logic
- Program dependencies
- Database access
- File access

The parser must extract metadata without attempting business interpretation.

Inputs

COBOL source files.

Instructions

Design a parser architecture capable of extracting:

Identification Division

- PROGRAM-ID
- AUTHOR

- INSTALLATION
- DATE-WRITTEN

Environment Division

- File assignments
- Configuration

Data Division

- Working Storage
- File Section
- Linkage Section

Procedure Division

- CALL Statements
- PERFORM Statements
- Paragraphs
- Sections

Embedded SQL

- Tables
- Views
- Operations

Generate:

- Architecture Diagram
- Modules
- Data Models
- Error Handling Strategy
- Testing Strategy

Expected Output

Parser Overview

Component Diagram

Metadata Extraction Flow

Error Handling Design

Testing Design

Constraints

Do not generate implementation code.

Focus on architecture and metadata extraction.

Success Criteria

Parser supports enterprise-scale COBOL repositories.

Example Usage

Design a parser architecture for a repository containing 10,000+ COBOL programs.

Review Checklist

- PROGRAM-ID extraction supported?
 - CALL extraction supported?
 - SQL extraction supported?
 - Fault tolerance defined?
 - Metadata model aligned?
-

Prompt 07

JCL Parser Architecture

File

prompts/parsing/jcl_parser.md

Purpose

Design a parser for Job Control Language.

Context

JCL defines:

- Execution flows
- Program execution
- Dataset usage
- Batch orchestration

Understanding JCL is critical for root-program identification.

Inputs

JCL source members.

Instructions

Design parser architecture capable of extracting:

Job Metadata

- JOB Statements
- Parameters

Execution Metadata

- EXEC Statements
- Programs
- Procedures

Dataset Metadata

- DD Statements
- Inputs
- Outputs

Scheduling Metadata

- Dependencies
- Conditions

Generate architecture and metadata model.

Expected Output

Parser Design

Metadata Model

Extraction Flow

Error Handling

Constraints

No implementation code.

Success Criteria

Every JCL job can be represented as metadata.

Example Usage

Design a parser for 5,000 production JCL jobs.

Review Checklist

- JOB extraction supported?
- EXEC extraction supported?
- DD extraction supported?

- Dependency extraction supported?
-

Prompt 08

Copybook Parser Architecture

File

prompts/parsing/copybook_parser.md

Purpose

Design parser architecture for enterprise copybooks.

Context

Copybooks define shared data structures across thousands of programs.

They often contain the most reusable business knowledge.

Inputs

Copybook source files.

Instructions

Support extraction of:

Structure Metadata

- Levels
- Field Names
- PIC Clauses

Complex Structures

- REDEFINES
- OCCURS
- OCCURS DEPENDING ON

Relationships

- Parent Child Hierarchies
- Reusable Structures

Generate parser architecture and metadata schema.

Expected Output

Parser Architecture

Metadata Model

Validation Strategy

Complexity Scoring Model

Constraints

No business interpretation.

Success Criteria

Complex enterprise copybooks represented consistently.

Example Usage

Design parser architecture for 20,000 copybooks.

Review Checklist

- PIC supported?
 - OCCURS supported?
 - REDEFINES supported?
 - Nested structures supported?
-

Prompt 09

DB2 SQL Parser Architecture

File

prompts/parsing/db2_sql_parser.md

Purpose

Design parser architecture for embedded DB2 SQL.

Context

SQL statements reveal:

- Data dependencies
- CRUD operations

- Data lineage
- Integration points

Inputs

EXEC SQL blocks.

Instructions

Support extraction of:

Objects

- Tables
- Views
- Aliases

Operations

- SELECT
- INSERT
- UPDATE
- DELETE

Access Patterns

- Joins
- Predicates
- Cursors

Relationships

- Program → Table
- Program → View

Generate metadata architecture.

Expected Output

Parser Architecture

Dependency Model

Metadata Schema

Validation Rules

Constraints

Focus only on metadata extraction.

Success Criteria

All SQL dependencies become graph-ready metadata.

Example Usage

Analyze DB2 usage across an enterprise card-processing application.

Review Checklist

- CRUD extraction supported?
 - Table extraction supported?
 - Join extraction supported?
 - Cursor extraction supported?
-

Prompt 10

VSAM Parser Architecture

File

prompts/parsing/vsam_parser.md

Purpose

Design parser architecture for VSAM metadata extraction.

Context

Many mainframe systems rely heavily on VSAM.

VSAM relationships are critical modernization inputs.

Inputs

COBOL source code and VSAM definitions.

Instructions

Support extraction of:

VSAM Objects

- KSDS
- ESDS
- RRDS

Access Methods

- READ
- WRITE
- REWRITE
- DELETE
- START

Relationships

- Program → VSAM
- Job → VSAM

Generate architecture and metadata model.

Expected Output

Parser Design

Metadata Schema

Relationship Model

Validation Rules

Constraints

Do not infer business semantics.

Success Criteria

VSAM usage becomes traceable and graph-ready.

Example Usage

Analyze VSAM dependencies in a core banking platform.

Review Checklist

- Access patterns captured?
- Dataset relationships captured?
- CRUD operations represented?
- Metadata graph ready?

Part 2 Completion Criteria

Upon completion of these prompts, MIP should be capable of converting raw source code into structured metadata.

MIP must be able to answer:

- Which programs exist?
- Which jobs execute them?
- Which copybooks define data?
- Which DB2 tables are accessed?
- Which VSAM files are used?
- Which dependencies exist?

Only after parsing is complete should metadata modeling begin.

End of Part 2.